Codebase Knowledge AI – Project Report

1. Introduction

In real-world software projects, codebases can become huge and confusing. When a developer joins a new project, understanding the code quickly becomes difficult. Searching using keywords doesn't always work because code is about meaning, not just words.

So, I built Codebase Knowledge AI, a system where you can ask questions in normal English (like "Where is login handled?"), and it finds the most relevant code for you.

In simple terms: It's like ChatGPT, but for understanding your codebase. ⍰

2. Objective

The main objectives of this project are:

- To simplify navigation within large codebases
- To enable natural language interaction with source code
- To improve developer productivity and efficiency
- To demonstrate the practical use of AI embeddings and vector search

3. Key Features

- Semantic code search based on meaning rather than keywords
- Ability to ask questions in natural language
- Fast retrieval of relevant code using FAISS
- Automatic parsing and chunking of code files
- Efficient embedding generation for similarity comparison

4. Technology Stack

- Python – Core programming language
- Sentence Transformers – For generating embeddings
- FAISS – For efficient similarity search
- NumPy – For numerical operations
- OS Module – For file handling

5. System Architecture

The system follows these steps:

1. The application reads all Python files from the project directory.
2. The code is split into smaller chunks (mainly functions).
3. Each chunk is converted into vector embeddings using a pre-trained model.
4. These embeddings are stored in a FAISS index for fast retrieval.
5. The user inputs a query in natural language.
6. The query is also converted into an embedding.
7. FAISS performs similarity search to find the closest matching code snippets.
8. The system returns the most relevant code segments as output.

6. Project Structure

- The project is organized as follows:
- app.py – Handles user interaction
- main.py – Contains core logic for embeddings and search
- my_project/ – Sample code files
  - auth_service.py
  - login.py
  - register.py

- logout.py
- utils.py
- requirement.txt – List of dependencies
- README.md – Project overview
- .gitignore – Ignores unnecessary files like pycache

7. Implementation Details

7.1 Code Loading

The system scans the project directory and reads all .py files using file handling methods.

7.2 Code Chunking

The code is split based on function definitions so that each function can be independently searched.

7.3 Embedding Generation

Each chunk is converted into vector form using the Sentence Transformer model (all-MiniLM-L6-v2).

7.4 Vector Storage

All embeddings are stored in a FAISS index to enable fast similarity search.

7.5 Query Processing

User input is converted into an embedding and compared with stored vectors to find relevant matches.

8. Advantages

- Faster understanding of complex codebases
- AI-based intelligent search
- More effective than traditional keyword search
- Useful for beginners and experienced developers
- Scalable to larger projects

9. Limitations

- Currently supports only Python files
- Basic chunking method (function-level only)
- No graphical user interface (CLI-based)
- No deployment or authentication features

10. Future Enhancements

- Support multiple programming languages
- Integrate advanced AI models for explanations
- Add user authentication and access control
- Deploy on cloud platforms

11. Conclusion

The Codebase Knowledge AI project demonstrates how Artificial Intelligence can enhance developer productivity by enabling semantic understanding of code. By combining embeddings and vector search, the system provides fast and meaningful results, making it a powerful tool for navigating modern codebases.

Author

Abhinaya Somishetty